

RoleMem: A Temporal Consistency Evaluation Framework for Role-Based Agent Memory

RoleMem: A Temporal Consistency Evaluation Framework for Role-Based Agent Memory

Abstract

Autonomous Large Language Model (LLM) agents deployed for long-horizon software engineering tasks rely heavily on episodic and factual memory systems to store observations about codebase architecture, function signatures, default configurations, and external dependencies. However, real-world software repositories evolve continuously. As codebases undergo refactoring, signature modification, parameter default shifts, and deprecations, static agent memory stores inevitably suffer from *epistemic obsolescence*—leading agents to retrieve stale facts, emit invalid API calls, and induce runtime failures. In this paper, we propose **RoleMem**, a framework and evaluation architecture designed to maintain temporal consistency in role-based agent memory across evolving codebases. RoleMem introduces: (1) a structured 6-tuple memory representation $\mathcal{M} = \langle c, \mathcal{E}, \mathcal{R}, \gamma, \tau, \Phi \rangle$ integrating structured claims, grounding provenance, epistemic roles, dynamic confidence, temporal anchors, and cryptographic integrity hashes; (2) role-aware invariant routing that dispatches memory verification across four epistemic roles (API, Configuration, Behavior, and Dependency); and (3) a dynamic three-state lifecycle engine ($\mathcal{A}$: PRESERVE, DOWNGRADE, INVALIDATE) that captures non-breaking evolutionary changes. To systematically evaluate temporal memory consistency, we introduce the **RoleMem Benchmark Protocol V2.2**, comprising 150 stratified claims extracted across 50 real-world repository transitions spanning 25 open-source Python repositories, adjudicated with dual independent annotations ($\kappa = 1.0$). On this benchmark, RoleMem achieves **100.0% Macro-F1**, reducing both False Invalidation Rate (FIR) and Stale Escape Rate (SER) to **0.0%** on the evaluated benchmark claims, outperforming static AST checkers (31.0% F1) and Naive RAG (28.7% F1). Component ablations demonstrate that removing epistemic roles ($-\mathcal{R}$), grounding evidence ($-\mathcal{E}$), or dynamic lifecycle modeling ($-\mathcal{A}$) degrades Macro-F1 by **-68.8%**, **-59.3%**, and **-67.5%**, respectively. Finally, an independent robustness challenge suite ($N = 30$) demonstrates RoleMem’s behavior under edge-case refactoring patterns while explicitly demarcating the operational boundaries of static invariant verification.

1. Introduction

Autonomous coding agents powered by Large Language Models (LLMs) represent a promising paradigm in software engineering automation, executing tasks ranging from automated bug localization and feature implementation to multi-repository dependency migration. To operate effectively across complex codebases without exceeding prompt context windows, modern agent architectures incorporate persistent external memory stores. These systems accumulate factual knowledge extracted during code exploration, such as callable signatures, default parameter values, framework configuration flags, and test behavior assertions.

Despite their utility in static environments, contemporary agent memory architectures suffer from a fundamental vulnerability: **epistemic obsolescence**. Software repositories are dynamic artifacts that evolve continuously through Git commit histories. Over time: - Functions and methods are renamed, relocated, or have their parameter lists altered. - Default configuration values are calibrated (e.g., changing a timeout default from 30 to 60 seconds). - APIs undergo soft or hard deprecation with scheduled sunset cycles. - Packaging dependencies and module imports are refactored.

When an agent consults a static vector database or key-value memory store populated in a prior repository state S_{base} , it retrieves obsolete factual assertions. If the current repository state S_{target} has modified these contracts, the agent exhibits *stale memory escape*—invoking non-existent arguments or violating newly introduced invariants. Existing approaches attempt to mitigate this through generic vector re-indexing or naive semantic similarity searches. However, semantic similarity is fundamentally limited when distinguishing fine-grained syntactic mutations: changing `timeout=30` to `timeout=60` produces near-identical text embeddings but represents a semantic parameter mutation that invalidates caller assumptions. Furthermore, repeatedly feeding raw multi-file diffs into frontier LLM prompts incurs substantial latency (multiple seconds per query) and token cost, making runtime per-step validation expensive.

To address this challenge, we introduce **RoleMem**, a framework and dynamic verification system for evaluating and maintaining temporal memory consistency in autonomous agents. Rather than treating memory as unstructured text chunks, RoleMem conceptualizes agent memory as grounded, role-differentiated epistemic invariants. Operating as an inline verification gate with an average latency of **22.6ms** per memory unit and zero LLM token overhead, RoleMem validates candidate memories prior to agent tool execution.

Key Contributions:

1. **Structured 6-Tuple Memory Representation:** We define a formal memory unit $\mathcal{M} = \langle c, \mathcal{E}, \mathcal{R}, \gamma, \tau, \Phi \rangle$, binding factual claims to concrete physical grounding provenance, epistemic roles, dynamic confidence scores, temporal anchors, and SHA-256 cryptographic integrity hashes.
2. **Role-Aware Epistemic Invariant Routing:** We define four distinct epistemic memory roles—*API*, *Configuration*, *Behavior*, and *Dependency*—and implement specialized AST invariant checkers (including the `DefaultValueEvolutionChecker` and unified deprecation visitors) that systematically evaluate evolution transitions.
3. **Dynamic Three-State Lifecycle Engine:** We model repository evolution beyond binary validity, implementing a transition function Λ with states `VALID (PRESERVE)`, `PARTIALLY_VALID (DOWNGRADE)`, and `STALE (INVALIDATE)` that allows agents to adapt to backward-compatible widening and soft-deprecations.
4. **RoleMem Benchmark V2.2 & Dual Gold Annotation:** We establish a preregistered benchmark of 150 stratified claims mined across 50 real-world evolutionary transitions from 25 canonical Python repositories, verified via dual independent human annotation with high inter-annotator agreement ($\kappa = 1.0$).
5. **Comprehensive Empirical Validation & Ablations:** We report full benchmark evaluation, answering three formal Research Questions (RQs) and demonstrating that RoleMem achieves 100.0% Macro-F1 with 0.0% false invalidations on the evaluated dataset, while ablations show that each core component provides between + 59.3% and + 68.8% absolute F1 improvement.
6. **Independent Robustness & Boundary Analysis:** We construct a 30-case independent robustness challenge suite to stress-test ungrounded retrieval, ambiguous evolution, and conflicting multi-channel metadata, explicitly defining the operational boundaries of static agent memory verification.

2. Related Work

2.1 LLM Agent Memory Architectures

External memory systems for LLM agents generally fall into episodic, semantic, and working memory paradigms (e.g., Generative Agents, MemGPT, AgentLite). Most implementations utilize Dense Passage Retrieval (DPR) or Approximate Nearest Neighbor (ANN) vector indices over

chunked text representations. While effective for thematic retrieval, these systems lack temporal grounding and type-aware semantics, making them prone to subtle evolutionary invalidations in formal code structures.

2.2 Software Repository Evolution & Mining

Software engineering research has long investigated software evolution, API breaking changes, and semantic versioning (SemVer) enforcement (e.g., Dig & Johnson, Xavier et al., Cossette & Walker). Static difference engines analyze syntactic deltas across library releases. However, prior work focuses on compiler diagnostics or maintainer dashboards, rather than runtime verification and lifecycle state transitions of autonomous agent memory stores operating under partial repository observability.

2.3 Epistemic Consistency, Knowledge Graphs, and Graph RAG

In knowledge representation, belief revision and temporal knowledge graphs study the maintenance of consistent assertions over time. Recent extensions like Graph RAG construct associative graph structures over text entities. However, associative graph representations still rely on natural language embeddings for edge traversal, leaving them vulnerable to fine-grained code invariant shifts. RoleMem bridges belief revision concepts with static program analysis, formalizing memory lifecycle transitions as dynamic state updates over grounded AST subtrees.

3. The RoleMem Framework

+-----+
-----+
|
|
+-----+
-----+

RoleMem Architecture Overview

1. Formal 6-Tuple Memory Representation:
M = < c, E, R, gamma, tau, Phi >
where:
c : Factual Claim (Structured slots + Natural Language Statement)
E : Grounding Evidence Provenance (File path, Line number, AST Snippet)
R : Epistemic Memory Role (API, Config, Behavior, Dependency)
gamma : Dynamic Epistemic Confidence Score in [0.0, 1.0]

tau : Temporal Anchor (Commit SHA, Release Ref, Timestamp)
Phi : Deterministic SHA-256 Cryptographic Fingerprint Integrity Hash

2. Role-Aware Semantic Invariant Routing:

- API Role -> Callable AST node, signature compatibility, soft/hard deprecation.
- Config Role -> DefaultValueEvolutionChecker (preserved, mutated, removed, required).
- Behavior Role -> Test suite assertion witness extraction & behavioral contracts.
- Dependency Role -> Packaging metadata (setup.py, pyproject.toml) manifest verification.

3. Dynamic Lifecycle State Transition Engine:

- PRESERVE (VALID) -> Boost confidence, advance temporal anchor tau -> S_target.
- DOWNGRADE (PARTIALLY_VALID) -> Decay confidence, attach non-breaking migration advisory.
- INVALIDATE (STALE) -> Zero confidence, purge from active agent working set.

3.1 Formal 6-Tuple Memory Unit

A memory unit $\mathcal{M} \in \mathbb{M}$ is defined as: $\mathcal{M} = \langle c, \mathcal{E}, \mathcal{R}, \gamma, \tau, \Phi \rangle$ where: - **Claim c** : A structured factual proposition consisting of a target symbol identifier, attribute key, asserted value, and canonical natural language description. - **Evidence Provenance $\mathcal{E}$** : Physical grounding coordinates $\mathcal{E} = \langle \text{file_path}, \text{line_number}, \text{ast_snippet} \rangle$. - **Epistemic Role $\mathcal{R}$** : Domain taxonomy $\mathcal{R} \in \{\text{API}, \text{CONFIG}, \text{BEHAVIOR}, \text{DEPENDENCY}\}$. - **Confidence γ** : Epistemic certainty $\gamma \in [0.0, 1.0]$. - **Temporal Anchor τ** : Commit hash S_{base} representing the temporal state where c was observed. - **Integrity Hash Φ** : Cryptographic SHA-256 digest computed over $\text{canonicalize}(c, \mathcal{E}, \mathcal{R}, \tau)$ to provide tamper-evident provenance across multi-agent handoffs.

3.2 Epistemic Memory Roles & Invariant Validators

1. **API Role ($\mathcal{R}_{\text{API}}$)**: Governs callable existence, positional/keyword parameter alignment, and deprecation status. The API validator parses the target AST node, extracting positional args, varargs (*args), kwargs (**kwargs), and keyword-only parameters. It inspects `@deprecated` decorators and `warnings.warn` statements.
2. **Configuration Role ($\mathcal{R}_{\text{CONFIG}}$)**: Governs parameter default values and configuration flags. The DefaultValueEvolutionChecker compares default AST expressions between S_{base} and S_{target} :

- Same parameter + same default $\implies$ VALID.
 - Same parameter + changed default $\implies$ PARTIALLY_VALID (backward compatible call, altered semantics).
 - Parameter removed $\implies$ STALE.
 - Parameter newly required without default $\implies$ STALE.
3. **Behavior Role ($\mathcal{R}_{\text{BEHAVIOR}}$):** Governs return types and runtime invariant contracts, grounded via test assertion witnesses and behavioral docstrings.
 4. **Dependency Role ($\mathcal{R}_{\text{DEPENDENCY}}$):** Governs package dependencies and version constraints, grounded in `setup.py`, `pyproject.toml`, and `requirements.txt`.

3.3 Dynamic Lifecycle Engine & Memory Compaction

When the repository transitions from S_{base} to S_{target} , the Lifecycle Engine evaluates the historical memory against the target state: - **PRESERVE ($\mathcal{I}_{\text{VALID}}$):** The claim remains factually intact. The temporal anchor is advanced ($\tau \leftarrow S_{\text{target}}$) and confidence is boosted ($\gamma \leftarrow \min(1.0, \gamma + 0.05)$). - **DOWNGRADE ($\mathcal{I}_{\text{PARTIALLY_VALID}}$):** The interface has undergone backward-compatible widening or soft-deprecation. Confidence decays ($\gamma \leftarrow \gamma \times 0.70$), and an advisory migration note is attached to the memory payload injected into the agent prompt. - **INVALIDATE ($\mathcal{I}_{\text{STALE}}$):** The claim is broken. Confidence is zeroed ($\gamma \leftarrow 0.0$) and the entry is evicted from active working memory into an archival audit log to maintain bounded memory capacity.

3.4 Scalability and Multi-Indexed Store Architecture

RoleMem maintains dual $\mathcal{O}(1)$ indexing over symbol names and file paths. Because AST parsing is scoped strictly to the specific source file referenced in provenance $\mathcal{E}$ rather than parsing the entire codebase, verification completes in sub-30ms times across repositories of varying scale (2k - 350k LOC).

4. Benchmark Protocol & Methodology

4.1 Benchmark Construction & Dataset Curation

RoleMem Benchmark Protocol V2.2 was preregistered and frozen to prevent experimental data leakage. The benchmark dataset comprises: - **25 Canonical Open-Source Repositories:** Selected

across diverse domains (web frameworks like Flask/FastAPI, data processing like Pandas/Dask, async runtimes, devops tools, and system utilities). - **50 Evolutionary Transitions** ($\mathcal{T} = \langle \mathbf{S}_{base}, \mathbf{S}_{target} \rangle$): Mined via AST delta filtering to ensure that each commit transition contains non-trivial signature changes, default parameter modifications, or dependency updates, excluding cosmetic edits. - **150 Stratified Benchmark Claims**: - SIGNATURE_COMPATIBLE: 50 claims (API Role) - DEFAULT_VALUE: 50 claims (Config Role) - DEPRECATION_STATUS: 28 claims (API Role) - BEHAVIORAL_CONTRACT: 17 claims (Behavior Role) - DEPENDENCY_CONTRACT: 5 claims (Dependency Role)

4.2 Dual Gold Annotation

To establish reliable ground truth, 75 complex claims were independently double-annotated by two senior software engineers under strict isolation protocol. Inter-annotator agreement achieved a high agreement score of **Cohen’s Kappa $\kappa = 1.0$** with zero label discordances.

4.3 Evaluation Tracks & Metrics

- **Track A (Strict 3-Class)**: Evaluates exact classification across VALID, STALE, and PARTIALLY_VALID.
- **Track B (Backward Compatible Binary)**: Evaluates binary usability (VALID + PARTIALLY_VALID vs STALE).
- **Primary Metrics**: Accuracy, Macro-Averaged F1, Per-Class F1, False Invalidation Rate ($\$FIR = \frac{\text{False STALE}}{\text{Total VALID}}\$$), and Stale Escape Rate ($\$SER = \frac{\text{False VALID}}{\text{Total STALE}}\$$).

5. Experimental Evaluation

5.1 Primary Results (Table 1)

Table 1 presents the comparative evaluation of RoleMem against baseline methods across all 150 benchmark claims.

Method	3- Class Acc	Macro-F1	Track A (Strict) Acc / F1	Track B (Compat) Acc / F1	FIR (False Inval) ↓	SER (Stale Escape) ↓	Avg Actions	Latency / Case
Baseline-1: Majority	83.3%	30.3%	83.3% / 45.5%	84.0% / 45.6%	0.0%	100.0%	0.0	0.0000s
Baseline-2: Static AST Checker	72.7%	31.0%	72.7% / 46.4%	73.3% / 46.7%	14.4%	91.7%	1.0	0.0059s
Baseline-3: Naive RAG	54.7%	28.7%	55.3% / 44.2%	54.7% / 42.9%	40.0%	70.8%	2.0	0.0039s
RoleMem (Ours)	100.0 %	100.0%	100.0% / 100.0%	100.0% / 100.0%	0.0%	0.0%	0.1	0.0226s

Latency Measurement Protocol: - *Hardware:* Linux 6.6 x86_64, 16 vCPUs (AMD/Intel), 64 GB RAM. - *Repository Scale:* Evaluated on 25 repositories spanning 2,000 to 350,000 LOC (50 to 4,200 Python source files per repo). - *Caching Protocol:* Pre-cached bare Git repositories residing on NVMe SSD storage (/tmp/formal_bare_repos/); measured under warm OS buffer cache following a 10-case warmup pass to accurately isolate computational and parsing latency. - *Repetitions:* 5 independent evaluation rounds across all $N = 150$ claims, yielding a mean verification latency of $22.6\text{ms} \pm 1.8\text{ms}$ per claim.

Key Findings: 1. **Mitigating Stale Memory Escape:** RoleMem reduces stale memory escape to $SER = 0.0\%$ on the evaluated benchmark claims, whereas baselines exhibit high escape rates (70.8% – 100.0%), allowing broken facts to persist. 2. **Preventing False Invalidation:** RoleMem maintains $FIR = 0.0\%$ on benchmark cases, whereas Naive RAG incorrectly invalidates 40.0% of valid memories due to ungrounded semantic drift. 3. **Execution Efficiency:** RoleMem verifies memory in **0.0226s per claim**, enabling practical inline verification in agent execution loops without LLM token expenditure.

5.2 Research Questions & Component Ablations (Table 2)

To systematically answer our three core research questions, we evaluate three ablation variants of RoleMem on the frozen benchmark.

Architecture Variant	Accuracy	Macro-F1	Δ F1	VALID F1 (Rec)	STALE F1 (Rec)	PARTIAL F1 (Rec)	FIR ↓	SER ↓
RoleMem (Full System)	100.0%	100.0%	—	100.0% (100.0%)	100.0% (100.0%)	100.0% (100.0%)	0.0%	0.0%
w/o Epistemic Roles ($-\mathcal{R}$)	73.3%	31.2%	- 68.8%	84.4% (86.4%)	9.3% (8.3%)	0.0% (0.0%)	13.6% %	91.7% %
w/o Grounding Evidence ($-\mathcal{E}$)	67.3%	40.7%	- 59.3%	77.1% (64.8%)	44.9% (83.3%)	0.0% (0.0%)	35.2% %	16.7% %
w/o Dynamic Lifecycle Engine ($-\Lambda$)	73.3%	32.5%	- 67.5%	84.6% (85.6%)	13.0% (12.5%)	0.0% (0.0%)	14.4% %	87.5% %

Answering Research Questions:

- **RQ1: Role-Aware Memory Specialization:** Removing epistemic roles ($-\mathcal{R}$) drops Macro-F1 by $-\mathbf{68.8\%}$ (to 31.2%). Without role dispatch, the system is unable to detect configuration default shifts and dependency constraints ($SER = 91.7\%$).
- **RQ2: Evidence Grounding:** Stripping grounding evidence ($-\mathcal{E}$) drops Macro-F1 by $-\mathbf{59.3\%}$ (to 40.7%), driving False Invalidation Rate up to **35.2%** due to global namespace collisions across polymorphic symbols.
- **RQ3: Dynamic Lifecycle Modeling:** Disabling the 3-state lifecycle engine ($-\Lambda$) drops Macro-F1 by $-\mathbf{67.5\%}$ (to 32.5%), failing to detect backward-compatible modifications (PARTIALLY_VALID Recall = 0.0%).

5.3 Epistemic Role Breakdown (Table 3)

Table 3 analyzes model accuracy and F1 score partitioned across the four epistemic memory roles.

Epistemic Role	Support (N)	Ground Truth Distribution	Majority Acc / F1	Static AST Acc / F1	Naive RAG Acc / F1	RoleMem (Ours) Acc / F1
API Role	78	Valid: 74 / Stale: 3 / Partial: 1	94.9% / 32.5%	97.4% / 59.6%	66.7% / 31.1%	100.0% / 100.0%
CONFIG Role	50	Valid: 29 / Stale: 21 / Partial: 0	58.0% / 36.7%	58.0% / 36.7%	60.0% / 52.4%	100.0% / 100.0%
BEHAVIOR Role	17	Valid: 17 / Stale: 0 / Partial: 0	100.0% / 100.0%	23.5% / 38.1%	0.0% / 0.0%	100.0% / 100.0%
DEPENDENCY Role	5	Valid: 5 / Stale: 0 / Partial: 0	100.0% / 100.0%	0.0% / 0.0%	0.0% / 0.0%	100.0% / 100.0%

RoleMem achieves consistent performance across all four roles on the standard benchmark, indicating that role-specific invariant routing addresses distinct facets of repository evolution.

5.4 Independent Robustness Challenge Suite (Table 4)

To stress-test RoleMem beyond standard repository patterns, we evaluate an independent suite of 30 edge cases across three challenge categories.

Challenge Category	Cases	Majority Acc / F1	Static AST Acc / F1	Naive RAG Acc / F1	RoleMem (Ours) Acc / F1
Evidence Missing (ROB-EM)	10	50.0% / 33.3%	50.0% / 33.3%	50.0% / 33.3%	50.0% / 33.3%
Ambiguous Evolution (ROB-AE)	10	40.0% / 19.1%	50.0% / 34.8%	50.0% / 31.6%	70.0% / 47.9%

Challenge Category	Cases	Majority Acc / F1	Static AST Acc / F1	Naive RAG Acc / F1	RoleMem (Ours) Acc / F1
Conflicting Evidence (ROB-CE)	10	100.0% / 100.0%	20.0% / 33.3%	20.0% / 33.3%	20.0% / 33.3%
Overall Robustness Suite	30	63.3% / 25.9%	40.0% / 26.4%	40.0% / 25.4%	46.7% / 30.1%

Diagnostic Insights: - **Ambiguous Evolution:** RoleMem achieves 70.0% accuracy on complex refactorings (variadic `*args/**kwargs` forwarding and nested wrappers), outperforming Majority (40.0%) and AST (50.0%). - **Evidence Missing:** When physical grounding is intentionally omitted, accuracy decreases to 50.0%, demonstrating that physical provenance is necessary to avoid heuristic namespace ambiguity. - **Conflicting Evidence:** Under contradictory signals (e.g., module docstring deprecated but AST active), RoleMem’s fail-closed policy conservatively flags `PARTIALLY_VALID` or `STALE` to prioritize agent safety.

6. Discussion & In-Depth Analysis

6.1 Understanding Standard Benchmark Performance

The high performance of RoleMem on the standard benchmark stems from its design as a **deterministic syntactic invariant validation over structured AST subtrees**, rather than an empirical statistical approximation over unstructured natural language. When: 1. Physical grounding $\mathcal{E}$ uniquely localizes the target entity, 2. Invariants are evaluated via domain-specialized AST visitors (`DefaultValueEvolutionChecker`), and 3. Code modifications are statically observable within the repository Git tree, RoleMem systematically validates the specified syntactic invariant over the parsed AST. The non-triviality of this task is substantiated by the severe failure of the baselines (Macro-F1 of 28.7% – 31.0%) and the substantial ablation drops when any single architectural component is removed (Macro-F1 dropping to 31.2% – 40.7%).

6.2 Failure Taxonomy and Boundary Conditions

Our robustness experiments illuminate the precise operational boundary between static AST analysis and dynamic runtime execution: 1. **Dynamic Metaprogramming:** APIs dynamically constructed via

`setattr` or `__getattr__` cannot be statically parsed without sandbox test witness execution. 2.

Variadic Forwarding: Functions delegating to internal `kwargs.get()` require inter-procedural dataflow analysis to confirm parameter acceptance. 3. **Multi-Channel Precedence:** When docstrings and AST decorators diverge, resolving developer intent requires multi-channel disambiguation.

7. Limitations & Operational Boundaries

1. **Dynamic Python Behavior:** RoleMem’s static invariant analysis operates on AST syntax nodes. Codebases relying extensively on dynamic metaprogramming (e.g., dynamic attribute assignment via `setattr()`, dynamic method dispatch via `__getattr__()`, or runtime monkey-patching) cannot be fully resolved statically without live test execution witnesses.
 2. **Missing Grounding Evidence:** When memory claims lack physical file provenance $\mathcal{E}$ (e.g. conversational memories generated without file anchors), RoleMem falls back to ungrounded heuristic search, which is susceptible to namespace collisions across identical helper function names.
 3. **Conflicting Multi-Channel Evidence:** In cases where documentation (docstrings), packaging metadata, and AST decorators provide contradictory status indications, RoleMem adopts a conservative fail-closed strategy that may flag non-breaking APIs as partially valid.
 4. **Observable Repository Evolution:** Invariant checking assumes transitions are observable via version-controlled repository files; external microservice state shifts or untracked environment variables remain outside the static analysis scope.
 5. **Ecosystem Scope:** The current reference implementation targets Python 3.8–3.13 source code and standard PEP packaging manifests; adapting to other ecosystems requires language-specific grammar extractors.
-

8. Conclusion

We presented **RoleMem**, a framework and evaluation architecture for maintaining temporal consistency in role-based agent memory across evolving software repositories. By integrating a structured 6-tuple memory schema, role-aware invariant routing, and dynamic 3-state lifecycle modeling, RoleMem mitigates stale memory escape and false invalidation with sub-30ms latency and without LLM token consumption. Systematic evaluation across 150 benchmark claims and 30 robustness edge cases validates our hypotheses and provides a principled empirical foundation for

building temporally consistent autonomous coding agents.